Ladda: Full Build Plan

For Kareem, Divine and Bello · Hackathon handoff · October 30, 2026

0. How to use this doc

This is the single source of truth for Ladda after a long planning chat. Paste it into any AI assistant and continue from Section 11 (Open decisions) and Section 10 (Build plan). Items marked PROPOSED were suggested but not yet confirmed by Kareem. Everything else was decided in the chat. Sections 13 to 15 hold the October 30 updates and win if anything above conflicts with them.

How Kareem wants an assistant to work:

- Informal, direct, practical. Step-by-step for academic problems.
- Confirm before generating large outputs. He is credit-conscious, so settle ideas and copy before building any page.
- Flag anything touching frontend, pages or design so Kareem and Bello decide together.
- Build as a sequential, division-by-division Claude Code workflow: one detailed prompt per division, confirm it works before the next.

1. Product in one page

Name: Ladda. **Tagline:** Pick right. Learn real. Get hired.

Problem: freshmen choose a course blindly. Final-year students with good GPAs still struggle to get hired because they lack industry skills and have no Plan B.

The loop: gap → roadmap → learn → build on real code → AI-reviewed project → Verified skill → signed credential an employer can check in seconds.

Positioning (PROPOSED): LORE for newcomers. At the GitLab AI Hackathon 2026, LORE won by preserving what senior engineers know, GraphDev won the Anthropic prize by mapping code, Gitdefender acted on its own inside code review, and Aegis won runner-up by explaining every decision. Lessons: agents that act beat chatbots, one specific user, AI as a multiplier, a demo that works first try, and always show the why.

Differentiator: the credential carries evidence (repo, commit, rubric scores), and only Verified skills reach employers.

2. Team, stack, workflow

Person	Lane
Kareem	AI: lib/ai, intake and roadmap API routes
Divine	Backend: prisma, db, credential
Bello	Frontend: globals.css, components, pages

Stack: Next.js 15, TypeScript, Tailwind v4, shadcn/ui, Framer Motion, Supabase (Postgres + Auth), Prisma, Vercel AI SDK, @noble/ed25519. AI provider is Google Gemini (switched from Anthropic for demo cost). Local folder: C:\Users\PROGRESSIVE\code\ladda-new.

Rough sketch only: a Claude Design canvas of the landing page exists (https://claude.ai/artifact/MJnbRGr731p7cXG1nef9q1). Kareem judged it not good enough. The real page is Next.js + Framer Motion.

3. Users and scope

- **Student**, three stages: Aspirant, Studying, Graduating.
- **Employer**: needs no account to verify a credential. Employers get no accounts. They verify from the landing page or /verify.
- **Departments (confirmed, four):** Computer Science, Electrical and Electronics Engineering, Computer Engineering, Information Systems or Data Science. Reason: the repo tutor and code tasks only fit software-adjacent paths, so every judge-tested user gets the full loop. Pitch line: starting with tech departments, expanding to every field.

4. Sitemap

Route	Access	Purpose
/	public	Landing, try-it AI demo
/auth	public	Sign up and sign in
/auth/verify-email	pending	Check your inbox, resend
/dashboard	student	Welcome animation, roadmap centerpiece, side rail

Route	Access	Purpose
/intake	student	Stage, target, optional CV, time
/roadmap	student	Full animated path; node panel opens on tap
/repo/[id]	student	Repo tutor: architecture map and Q&A
/credentials	student	Own credentials
/notes, /leaderboard, /opportunities, /opportunities/[id], /challenges	student	Dashboard extras
/verify, /verify/[id]	public	Employer verification, no login

Main flow: Landing → Sign up → Verify email → Dashboard welcome → Intake → Roadmap → Node (Learn, Build, Test) → Verified → Credential → Verify page.

5. Page flows

5.1 Landing

Headline (leaning): Pick right. Learn real. Get hired. Two problem cards (Freshman, Final year), bridge line: Your degree is the start. Ladda is the climb. A try-it demo with a toggle (choosing a course / about to graduate), one text box with example chips, streamed answer: fit % bars in course mode, skill-gap bars plus Plan B paths in graduate mode. Needs a real model call, rate limit and cached fallback. Then how-it-works (4 moves), an employer verify box (no login), a Meet the builders section, final CTA. Open: hero animation idea and tone.

5.2 Sign up and sign in

- Fields: name, university, department (dropdown of the four), email, password. Aspirants enter the university they are aiming for.
- Email verification before the dashboard unlocks. Risk: Supabase default email limits. Use a custom sender (Resend) and keep one pre-verified demo account.
- Email and password only. Google sign-in is dropped (decided October 30).
- First dashboard load plays Welcome to Ladda, [name] once (profiles.welcomed flag).

- The Complete your profile to continue gate is simply Intake being done. No separate profile page.

5.3 Intake (three questions, about 30 seconds)

1. **Stage:** Aspirant, Studying, Graduating.
2. **Target:** pick a role from the department list. Aspirants type what they enjoy and the AI ranks paths with fit %.
3. **Time:** hours per week and a horizon (4, 8, 12 weeks or until graduation).

Optional step between 2 and 3: **CV upload (PDF or DOCX, about 5MB, skippable).** Extract text (pdf-parse, mammoth), strip contact details, AI returns skills with the CV line as evidence (Zod-validated), map to catalog skill IDs using a synonyms field, show a review checklist (untick wrong ones), then matched nodes flip to Known with an animation and a banner like Your CV saved you 24 hours. Do not store the raw file by default. Scanned PDFs with no text fall back to manual I already know this toggles.

Then a Building your path animation leads into the roadmap.

Time planner (code, not AI): each skill has estimated hours and priority. Total hours divided by hours per week gives weeks. If it exceeds the horizon, must-haves stay and the rest become stretch nodes. The AI only writes the why and picks the practice task.

5.4 Dashboard

Centre: the roadmap path. Left rail (bottom tab bar on phone): Roadmap, Notes, Leaderboard, Opportunities, Credentials. Right edge: Next up card, two-tone progress ring (solid = Verified, light = Known), weeks left.

- **Notes:** attached to a node, plain text. AI hook: Quiz me on my notes.
- **Leaderboard:** ranks verified progress only. Filter by university and department. Seed with clearly labelled sample users so it is not empty on demo day.
- **Competitions:** now called Opportunities, with two tabs. Opportunities: at least 6 hand-added competitions and internships as cards with a detail view and an Apply link to the external page. Ladda challenges: an in-app weekly challenge shown as a coming-soon card (Check back Sunday). See Section 13.
- Priority: Must = roadmap, node flow, credential, verify. Cheap = notes and the opportunities cards. If time allows = leaderboard.

6. Roadmap engine

Node states: todo, known, in_progress, verified. Locked is a visual computed from prerequisites, never stored.

- **Known:** from CV or manual. Skipped from the time plan. Does not count for leaderboard, credential or employer page.
- **In progress:** started the node. Hours still count.
- **Verified:** project approved by the AI reviewer. The only state that counts.

Transitions: todo → in_progress (opens task); todo → known (CV or manual); known → verified (project submitted and approved); in_progress → verified (project approved).

Unlock rules (strict, enforced on the server):

1. A node unlocks when all prerequisites are finished (Verified or Known).
2. Phases run Foundations → Core → Build → Proof. The next phase opens when every must-have node in the previous one is finished. Stretch nodes never block.
3. Inside a node: Learn first; Build unlocks after Learn is done or a certificate is attached. Known nodes (from the CV) have Learn and Build both unlocked from the start. Verified comes from an approved project. The quiz step is a stretch goal.
4. **Anti-cheat (confirmed):** Known unlocks nodes in between, but the final Proof node and the credential need every must-have node Verified. Known skills must be verified by submitting a project first.

Details:

- A locked node is never a dead tap: it says what to finish first and jumps to it.
- When the last prerequisite becomes Verified, the path line lights up and dependants open one by one.
- Failing never relocks anything. Rejected projects get feedback and resubmit. Failed tests have a retake cooldown.
- Undo Known only if no dependant node has started.
- Progress is per skill, so changing target role keeps shared skills.
- Catalog validation script: no cycles, no orphan nodes.
- Demo: seed an account mid-path, labelled honestly.

7. Node verification (inside the node panel)

Tap a node to open the panel. Two steps are built for the hackathon; the third is a stretch goal:

1. **Learn:** hand-curated resources tagged Free or Paid, plus Study it in a real repo (repo tutor). The AI never invents URLs. It only ranks resources for the student's time and explains why.
2. **Build:** an assigned project plus 2 or 3 suggestions. Student submits a GitHub link. The AI reviewer reads the repo through the GitHub API and scores it against a rubric.
3. **Test:** STRETCH, not built for the hackathon: 5 to 8 questions generated from the skill rubric (multiple choice plus one explain-it answer), mostly graded in code, 70% pass, retake cooldown. PROPOSED cheaper stand-in: before approving a project, the AI asks two short questions about the student's own code.

Certificates are optional evidence, not a step. Upload a PDF or image, or paste a verification link. A certificate can skip the Learn step and is shown on the credential, but it never makes a node Verified. Known nodes only need an approved project.

Human review: pitch it as phase two (a mentor queue for appeals). Do not claim it on stage unless it is built.

8. Credential and verification

- Record: candidate name, skill, task, repo, commit hash, rubric scores, certificate if any, issue date, signed with ed25519. Private key lives in server env only. Public key served at an endpoint.
- **/verify/[id]:** big Valid, Invalid or Revoked status, candidate card, verified skills with scores, expandable evidence, plain-language summary written by the AI, and a Verify in your browser button that re-checks the signature client-side.
- **Tamper test** demo button: edit a score and watch Valid flip to Invalid.
- Employers see Verified skills only. Revoked flag supported.
- Honest limit, stated on the page: the signature proves Ladda issued the record and it was not altered; it does not prove the AI grading was perfect, which is why evidence links are open.

9. Architecture

Frontend (Bello, with Kareem on design calls): Next.js app router, Tailwind v4, shadcn/ui. Framer Motion for: path line drawing in, nodes popping in, active node pulse, unlock sequence, Known flip, welcome animation, streamed AI text, bars filling. React Flow for the repo architecture map (TS and JS repos only).

Backend (Divine): Supabase Auth with @supabase/ssr (cookie sessions), Prisma on Postgres, row-level security on profiles and user data, Supabase Storage for certificates (size and type limits). Middleware routes users by role. Prisma cannot manage Supabase auth tables, so keep a profiles table keyed to the auth user ID.

API routes (gist):

- POST /api/intake/suggest-paths, /api/intake/parse-cv, /api/intake/build-roadmap
- GET /api/roadmap; POST /api/node/[id]/start, /known, /project, /certificate (the test routes are stretch)
- POST /api/repo/analyze, /api/repo/ask
- POST /api/credential/issue; GET /api/verify/[id], /api/keys/public
- POST /api/notes; GET /api/leaderboard

AI layer (Kareem, lib/ai): modules for suggest-paths, parse-cv, roadmap explanations, review-project, summarise-credential (generate-test and grade-test are stretch), repo Q&A. Rules: every output validated against a Zod schema, a cached fallback when it fails, a rate limit to control Gemini cost, numbers computed in code and the model only explains.

Data model (main tables):

- profiles(id, role, display_name, university, department, stage, target_role_id, hours_per_week, horizon_weeks, welcomed, profile_complete)
- roles, skills(name, synonyms, hours), role_skills(role, skill, phase, priority, is_stretch), skill_prereqs(skill, prereq)
- user_nodes(user, skill, status, source, evidence_snippet)
- node_evidence(type: resource, project, certificate, test; url or file; status; score; reviewer: ai or human)
- resources(skill, title, url, cost), projects(skill, brief, rubric), tests and test_attempts (stretch), opportunities
- repos(url, analysis_json cached), credentials(payload, signature, issued_at, revoked), notes, challenges

Shared contract (freeze in the first 2 hours):

```
type Stage = 'aspirant' | 'studying' | 'graduating'
type NodeStatus = 'todo' | 'known' | 'in_progress' | 'verified'
type EvidenceType = 'resource' | 'project' | 'certificate' | 'test'
type Source = 'cv' | 'manual' | 'task'
```

Catalog spec (every node comes from it): per role, a list of skills with phase, hours, priority, prerequisites, synonyms, hand-curated resources, project template with rubric, and test seeds. First pilot role: Backend engineering, about 12 nodes across Foundations, Core, Build, Proof. Not yet drafted.

10. Build plan (3 days, 3 people)

Principles: freeze the contract first; build a vertical slice on day 1 (one repo, one task, one grade, one credential, deployed ugly on Vercel); deploy on every merge; cache the demo path (pre-analyse 2 or 3 repos, cached AI answers); feature freeze at the start of day 3; cut anything not in the 90-second demo.

Divisions, in order (each confirmed working before the next):

1. D0 Contract and catalog (all three; Backend pilot role)
2. D1 Auth, profiles, email verification (Divine, Bello)
3. D2 Intake and time planner, optional CV parse (Kareem, Divine, Bello)
4. D3 Roadmap graph and unlock engine, server-enforced (Divine, Kareem)
5. D4 Node flow: learn, build with AI project review; the test is stretch (Kareem, Divine, Bello)
6. D5 Repo tutor map and Q&A (Kareem, Bello)
7. D6 Credential signing and verify page (Divine, Kareem, Bello)
8. D7 Dashboard extras: notes, opportunities cards and detail view, Hacktoberfest popup, then leaderboard
9. D8 Landing page and polish (Bello, with Kareem)

11. Open decisions

1. Departments: closed. The four are confirmed.
2. Google sign-in: closed. Dropped; email and password only.
3. Competitions: closed. Hand-added opportunity cards now; the in-app challenge shows Check back Sunday.

4. Anti-cheat rule: closed. Confirmed.
5. Landing headline and hero animation (ladder building on scroll, or live typing AI).
6. Employer accounts: closed. Not building them.
7. Still open: draft the Backend engineering catalog, and propose roles for the other three departments (next task).

12. 90-second demo

Student with a backend target → Intake → roadmap draws in with a CV-matched node flipped to Known → open a node → repo tutor map → submit project → AI feedback and approval → node turns Verified and the next nodes unlock → credential issues → open the verify page on a phone → tamper test flips it to Invalid.

13. October 30 updates (these take priority)

Ten decisions are locked and the CV and verification flows changed. If earlier text conflicts with this section, this section wins.

- **Sign-up:** email and password only, with email verification. No Google.
- **Departments:** the four are confirmed.
- **Employers:** no accounts. They verify from a box on the landing page or at /verify.
- **CV flow:** after upload and extraction, matched skills become Known nodes. Tapping a Known node does not show a Start button or a locked Build step. Learn and Build are both unlocked at once. The student submits a project, and when the AI reviewer approves it the node becomes Verified. Hackathon scope stops here.
- **Verification rule:** Verified = approved project. The quiz is a stretch goal. PROPOSED: use the same project-first rule for every node, not only Known ones, so there is one rule to build and explain.
- **PROPOSED check:** a GitHub link can be copied, so before approval the AI asks two short questions about the student's own code. It is a cheap stand-in for the test.
- **Opportunities (replaces Competitions):** Kareem adds at least 6 competitions and internships by hand. Each shows as a card (title, organiser, type, deadline, short blurb) and opens a detail view (description, who can apply, requirements, dates, and an Apply button that opens the external link in a new tab). Filter by type: Competition or Internship. Cards come from a seeded opportunities table, so no admin screen is needed.

- **AWS Hacktoberfest popup:** after sign-in, once Intake is finished, the first dashboard view shows a popup: AWS HACKTOBERFEST IS LIVE. Check it out now. Two buttons: Cancel, and Look into it. Look into it opens the Hacktoberfest detail page in Opportunities. Cancel dismisses it for good. It shows once per student and only inside a date window (featured_from, featured_until on the opportunity), so it disappears by itself when the event ends.
- **Ladda challenges (internal competitions):** a coming-soon card on the Opportunities page, rendered at /challenges: Check back Sunday. Optional: a countdown to Sunday, computed in the browser.
- **Landing page:** add a Meet the builders section. The cards are driven by one config file (content/builders.ts with name, role, photo, bio, links) that Kareem fills in later while coding. Plan for three cards.

New data and routes: opportunities(id, title, organiser, type, deadline, description, requirements, apply_url, image, tags, featured_from, featured_until); profiles.announcement_dismissed; GET /api/opportunities, GET /api/opportunities/[id], POST /api/announcements/dismiss.

14. How the pages link together

Every page is reached through one main action, and a middleware guard decides who may see what.

From	Action	To	What travels
Landing (/)	Get started	/auth, sign-up tab	nothing
Landing	Sign in	/auth, sign-in tab	nothing
Landing	Enter a credential ID in the verify box	/verify/[id]	credential ID
Landing	Try-it demo	stays on page	text sent to POST /api/demo (public, rate limited)
Sign-up	Submit	/auth/verify-email	pending session
Verify-email	Click the link in the email	/auth/callback, then /dashboard	session cookie

From	Action	To	What travels
Sign-in	Success	/dashboard, or /intake if the profile is incomplete	session cookie
Dashboard	Complete your profile card	/intake	nothing
Intake	Finish	/dashboard with the roadmap built	stage, target role, time, optional CV skills
Dashboard	Hacktoberfest popup, Look into it	/opportunities/[id]	opportunity ID
Dashboard	Tap a node	node panel at /roadmap?node=ID	node ID
Node panel	Tap a locked node	explains the blockers and jumps to the prerequisite	blockedBy IDs
Node panel	Study it in a real repo	/repo/[id]	skill ID, repo ID
Node panel	Submit project	stays, shows the review result	GitHub URL
Roadmap	Proof node approved	/credentials/[id]	credential ID
Credential	Share link or QR	/verify/[id]	credential ID
Left rail	Notes, Leaderboard, Opportunities	/notes, /leaderboard, /opportunities	nothing
Opportunities	Tap a card	/opportunities/[id]	opportunity ID
Opportunity detail	Apply	external site, new tab	apply_url
Opportunities	Ladda challenges tab	/challenges, Check back Sunday	nothing

Route guard (middleware), checked in this order:

1. Public routes: /, /auth, /verify, /verify/[id], /api/demo, /api/verify/[id].

2. Not signed in anywhere else: send to /auth.
3. Signed in but email unverified: send to /auth/verify-email.
4. Email verified but profile_complete is false: send to /intake.
5. Signed in and visiting /auth: send to /dashboard. Everything else is open.

15. How frontend, backend and AI work together

The server owns every decision (status, unlocks, scores, signatures); the frontend only shows what the server returns.

The contract

- A shared package (lib/contract) holds the enums, a Zod schema for every request and response, and the inferred TypeScript types. The frontend calls typed wrappers in lib/api, and the routes validate with the same schemas. A contract change goes through one PR that all three review.
- A mock layer (fixtures in lib/mocks, switched on with NEXT_PUBLIC_USE_MOCKS=1) lets Bello build every page on day 1 before the routes exist.
- Every response is { ok: true, data } or { ok: false, code, message }. Codes: UNAUTHENTICATED, EMAIL_UNVERIFIED, PROFILE_INCOMPLETE, NODE_LOCKED (with blockedBy), VALIDATION, AI_FAILED, RATE_LIMITED. The UI maps each code to a state: skeleton, locked explanation, or retry.
- The roadmap payload is pre-computed. Each node arrives with status, locked, blockedBy and nextStep, so the UI never re-implements unlock rules. The rule engine (lib/roadmap/engine.ts) is pure functions with unit tests, owned by Divine. The AI never changes a status.

Request lifecycle: submitting a project

1. The UI posts the GitHub URL to POST /api/node/[id]/project.
2. The route reads the user from the session cookie (never a client-sent ID), checks engine.canBuild, and validates the URL.
3. The route writes a node_evidence row with status pending and returns 202 with an evidenceId.
4. lib/ai reviewProject reads the repo through the GitHub API, scores it against the rubric, and returns Zod-validated JSON. If the model fails, a cached fallback result is used and flagged.

5. The result is stored. Above the pass mark, the engine marks the node verified, recomputes unlocks, and returns the unlocked node IDs.
6. The UI polls GET /api/node/[id]/evidence/[evidenceId] every 2 seconds (up to 60 seconds), then shows the feedback and plays the unlock animation for the returned IDs.

Intake build-roadmap, CV parsing and credential issuing follow the same pattern. The server signs credentials, and the private key never leaves it. Opportunities are read-only.

Who does what

- **Frontend (Bello):** pages, animation and forms. Optimistic updates only for notes and the I already know toggle, never for verification.
- **Backend (Divine):** auth, middleware, database, row-level security, the rule engine, storage, signing, polling endpoints, seed scripts.
- **AI (Kareem):** everything in lib/ai, called only from server routes. The Gemini key stays server-side. Streaming (Vercel AI SDK) is used only for the landing demo and repo Q&A.

Shared setup

- Auth: @supabase/ssr cookies, middleware guards (Section 14), row-level security as the second layer.
- Seeds: a Prisma seed script loads roles, skills, prerequisites, resources, project templates, at least 6 opportunities, demo accounts and sample leaderboard users. The builders config stays a code file.
- Environment variables: NEXT_PUBLIC_SITE_URL, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, DATABASE_URL, GEMINI_API_KEY, GITHUB_TOKEN, RESEND_API_KEY, CREDENTIAL_PRIVATE_KEY, CREDENTIAL_PUBLIC_KEY.
- Branching: one folder per lane, PRs into main, a Vercel preview on every PR.

16. Handoff prompt

Paste this into the next assistant: I am continuing a hackathon project called Ladda. Read the attached plan fully. Do not rebuild or redesign decided items. Start with Section 11 (open decisions), ask me one question at a time, and confirm before generating any large output or building any page.